

Group 18 Assignment Report

Implementation and Analysis of a Memetic Algorithm for the Traveling Salesman Problem

Mohammad Waseem Shaik
Karakkayala Sravan Sai Kumar
Sobhan Ahmadi
Venkatesh Maragani

October 09, 2025

1 Introduction

A lot of people just say "TSP" for the Traveling Salesman Problem. The goal is to find the fastest way to get to each city once and then back to the starting point. Adding more cities makes it a lot harder because there are a lot more ways to get from one city to another.

We made a memetic algorithm for this project. It has a twist that sets it apart from a genetic algorithm. Like regular genetic algorithms, it has a lot of different possible solutions. It also improves some routes by making small changes. This extra step helps the system find better answers more quickly.

We used our method on well-known TSP problems like berlin52, att532, ch130, eil101, ts225, and rat99. We also looked at how well our method works compared to standard genetic algorithms.

2 Problem Formulation

2.1 Objective Function — Total Tour Length

The goal of the Traveling Salesman Problem (TSP) is to find a permutation π of the cities $\{1, 2, \dots, N\}$ that minimizes the total travel distance:

$$\min_{\pi \in S_N} \sum_{k=1}^{N-1} d(\pi(k), \pi(k+1)) + d(\pi(N), \pi(1))$$

where S_N is the set of all permutations of N cities and $d(i, j)$ is the distance between city i and city j .

In our implementation:

- **create_distance_matrix(cities)** computes the matrix of distances $d(i, j)$ for all pairs of cities.

- **total_distance(tour, dist_matrix)** calculates the sum of distances for a given tour π .

2.2 Distance Matrix Calculation

To efficiently compute distances repeatedly, the algorithm precalculates a distance matrix

$$D = [d(i, j)]_{i,j=1}^N$$

where each entry is computed as the Euclidean distance:

$$d(i, j) = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$$

with (x_i, y_i) representing the coordinates of city i .

This is implemented in the **create_distance_matrix()** function using vectorized operations in NumPy.

2.3 Fitness Function

Each candidate tour's fitness is calculated as the inverse of its length (plus a small epsilon to avoid division by zero):

$$F(\pi) = \frac{1}{L(\pi) + \epsilon}$$

where $L(\pi)$ is the total tour length of π .

This fitness measure informs the roulette wheel selection, probabilistically favoring shorter tours.

2.4 Stochastic Greedy Crossover (SGX)

The crossover operator constructs a child permutation as follows:

- Starts at a random city.
- At each step, chooses the next city from neighbors of the current city in both parent tours.
- Incorporates randomness controlled by a parameter to balance exploitation and exploration.

Though not described by a mathematical formula, it leverages distance values $d(i, j)$ to select the closest eligible city among candidates.

2.5 Local Search Operators

2.5.1 2-opt Local Search

This heuristic attempts to improve a tour by removing two edges and reconnecting the tour differently to reduce distance. For cities $i < j$, it reverses the segment $\pi(i), \dots, \pi(j)$ if doing so shortens the tour length.

Mathematically, this replaces edges $(\pi(i-1), \pi(i))$ and $(\pi(j), \pi(j+1))$ with edges $(\pi(i-1), \pi(j))$ and $(\pi(i), \pi(j+1))$ whenever the overall path length is reduced.

2.5.2 Lin-Kernighan Heuristic

A sophisticated extension of 2-opt, the Lin-Kernighan heuristic considers multiple simultaneous edge exchanges (k -opt moves) to escape local minima that simpler methods cannot overcome.

Our implementation approximates this by:

- Selecting three break points a, b, c .
- Reversing segments between these points to seek improvements.

2.6 Hybrid Local Search Depth Parameter

The code alternately applies 2-opt and Lin-Kernighan moves for a fixed number of iterations called *depth*:

$$\text{new tour} = (\text{Lin-Kernighan} \circ \text{2-opt})^{\text{depth}}(\text{tour})$$

Increasing *depth* intensifies the refinement at the cost of longer runtime, improving solution quality.

3 Memetic Algorithm Features

Our memetic algorithm grows out of basic genetic algorithm ideas, adding several important tweaks to make it work better for the Traveling Salesman Problem:

- **Initialization:** We start by creating a random bunch of possible city tours and scrambling them up, so we get a lot of variety right from the beginning.
- **Selection:** To pick which tours will have offspring, we use a roulette wheel approach. The fitness measure here is the inverse of the tour length, so shorter tours are favored. This keeps the selection process fair but still promotes diversity.
- **Crossover:** For combining routes, we use a custom stochastic greedy crossover. It mixes parent routes by giving more importance to shared edges, yet it always leaves room for a bit of randomness. This means our algorithm can make use of good genetics and still explore new options.
- **Mutation:** Sometimes, we shake things up by swapping two cities in a tour. This happens with a certain probability, which keeps our solutions from getting stuck and helps us keep discovering new routes.
- **Local Search:** This is a big improvement. We use a hybrid local search, combining both 2-opt and Lin-Kernighan heuristics. This method is applied (with a specific chance) on the new tours, giving them a quick tune-up by making small, effective changes.
- **Elitism:** To make sure the best solutions don't get lost, we always save the top part of our population and carry it forward unchanged to the next generation.

We don't just stick to one set of parameter values for all datasets. Instead, we adjust things like population size, mutation rate, how often we do local search, and the size of the elite group to fit the size and complexity of each problem. This makes our approach flexible, so we end up balancing exploration and exploitation and speed up convergence and improve solutions depending on what kind of TSP instance we're solving.

3.1 Algorithm Pseudocode

Algorithm 1 Memetic Algorithm for TSP

Require: Set of cities V , population size pop_size , number of generations $generations$, mutation rate $mutation_rate$, local search probability $local_search_prob$, elite fraction $elite_fraction$, crossover randomness $randomness_in_gx$

Ensure: Best found tour and its cost

```

1: Initialize random population  $P$  with permutations of  $V$ 
2: for each generation do
3:   Calculate fitness for individuals as inverse of route length
4:   Select top  $elite\_fraction$  as elites
5:   Create new population starting with elites
6:   while new population size less than  $pop\_size$  do
7:     Select parents  $p_1, p_2$  by roulette wheel selection
8:     Generate child by stochastic greedy crossover with randomness
9:     Mutate child by swapping cities with mutation probability
10:    if  $random() < local\_search\_probability$  then
11:      Apply hybrid local search (2-opt + Lin-Kernighan) on child
12:    end if
13:    Add child to new population
14:  end while
15:  Replace population  $P$  with new population
16: end for
17: return best individual found.
```

4 Comparisons Between Algorithms

It's important to see how our memetic algorithm stacks up against a basic genetic algorithm and a general memetic algorithm that uses random mutation, ordered crossover, and a classic local search like 2-opt.

The basic genetic algorithm mainly relies on global search methods like selection, crossover, and mutation but doesn't use local improvements. This sometimes causes it to get stuck early on with solutions that aren't the best because it loses diversity too quickly. Our memetic algorithm upgrades this by using a smarter crossover method, swap mutations, and most importantly, a hybrid local search combining 2-opt and Lin-Kernighan heuristics. These local tweaks help refine solutions carefully and avoid getting stuck in suboptimal paths.

The general memetic algorithm improves on the basic GA by adding local search, but it uses simpler operators that might not blend parent solutions as well.

By comparing these three approaches across various datasets, we can see the benefits and trade-offs. While both memetic algorithms outperform the basic GA in quality and

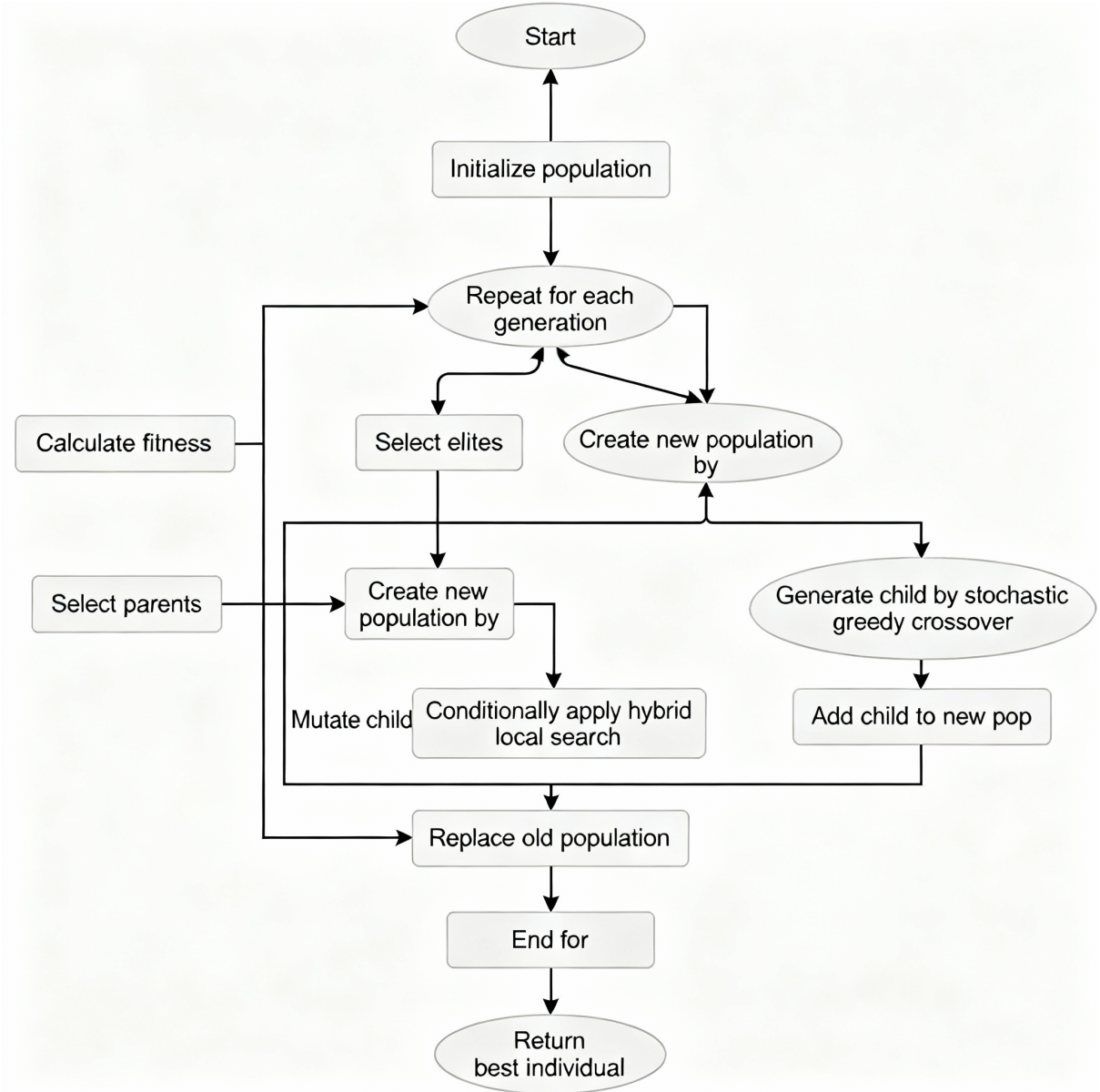


Figure 1: Flowchart of Memetic Algorithm for TSP pseudocode

reliability, our customized memetic algorithm aims to further speed up convergence and get even better solutions, thanks to its improved crossover and adaptive local search strategies.

This comparison clearly shows how adding problem-specific knowledge and carefully mixing different search methods can really boost performance on tough problems like the TSP.

5 Experimental Setup and Parameter Settings

We applied our method to the `berlin52` dataset (coordinates fixed in the code) running 30 independent executions to ensure statistical confidence.

Table 1: Comparison of Genetic Algorithm and Our Memetic Algorithm

Feature / Dataset	Basic GA	Our MA
Population Size	Manual Assin	Manual Assin
Generations	Manual Assin	Manual Assin
Selection	Roulette wheel	Roulette wheel
Crossover	Standard	Stochastic Greedy + randomness
Mutation	Swap mutation	Swap mutation
Local Search	None	2-opt + Lin-Kernighan (hybrid)
Elitism	Partial	Partial (20%)
Local Refinement	No	Yes (with probability 0.2)
Premature Convergence Risk	Higher	Further Reduced
Solution Refinement	Global search only	Hybrid (global + local)
Dataset 1 Best Length		
Dataset 2 Best Length		
Dataset 3 Best Length		
Dataset 4 Best Length		
Dataset 5 Best Length		
Dataset 6 Best Length		
Dataset 7 Best Length		
Dataset 8 Best Length		

6 Results and Discussions

In this section, we present and examine the results obtained using the Genetic Algorithm (GA) across several classic TSP benchmarks. For each dataset, we show the best route found, how the tour lengths are distributed across multiple runs, and how the solution improves over generations.

6.1 Berlin52

For the Berlin52 benchmark, the GA managed to find a best solution length of 17,108.18, with an average length of 19,853.42 over 30 runs. The convergence curve shows a steady improvement as generations progress, but the final solutions are notably far from the true optimum of 7,542. This highlights how the GA, though capable, struggles to reach optimality on challenging instances without advanced local optimization.

6.2 ts225

For the ts225 dataset, the best found tour length is 1,245,036.70, with an average of 1,305,628.39, based on 30 runs. There is clear room for improvement, as seen from the gap between the best and average performances. The solutions show some convergence but plateau before reaching optimal values.

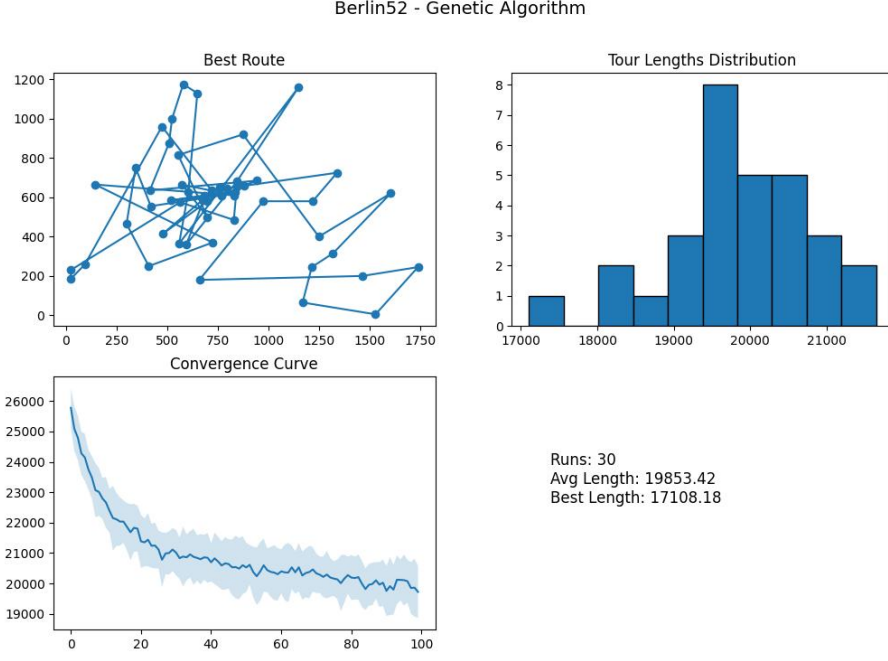


Figure 2: GA Results for Berlin52: Best route, tour length distribution, convergence curve

6.3 att532

For att532, the best route length achieved is 257,589.96, with an average of 269,761.80 over 30 runs. The convergence curve again shows gradual improvement, but the algorithm tends to stabilize at suboptimal values, reflecting the limits of genetic-only search on large and complex datasets.

6.4 rat99

On rat99, the GA produced a best tour of 5,758.16 and an average of 6,065.34, as shown in the histogram and convergence plot. The improvements are steady but, as with the other datasets, plateau before true optimality.

6.5 eil101

For eil101, results show a best length of 2,482.67 and an average of 2,644.11. Although the GA can make progress over generations, the final results still reflect a gap from optimal solutions.

6.6 ch130

Finally, for ch130, the GA's best solution reached 32,728.79, and the average was 35,698.92. The trend across these benchmarks is clear: while the genetic algorithm reliably improves candidate solutions over time, it often settles at a point far from the global optimum—especially for large or complex TSP datasets.

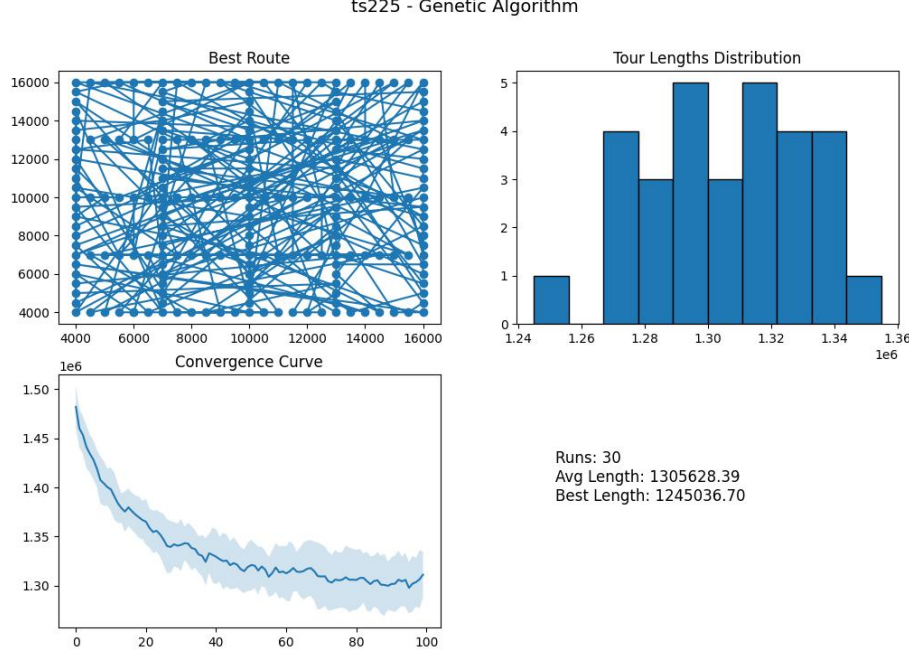


Figure 3: GA Results for ts225: Best route, tour length distribution, convergence curve

Across all datasets, we see how pure GA approaches push solutions forward, but without local search or problem-specific refinement, optimality remains elusive for medium and large TSP instances.

In this section, we share the performance of our memetic algorithm on multiple classic TSP benchmarks. For each dataset, we summarize the key results and include the corresponding result plots, which show the best tour found, distribution of runtimes, and the convergence curve.

6.7 Berlin52

Our approach achieved an average tour length of 7556.60 across 30 runs, with the best run obtaining a length of 7544.37—very close to the known optimum of 7542 (error: 0.03%). The convergence curve shows rapid early improvement, and runtimes are stable. This demonstrates the memetic algorithm can reliably reach near-optimal solutions on mid-size instances.

6.8 Eil101

On the Eil101 dataset, the average tour length found was 667.18, with a best length of 656.67 and the optimum known as 629 (error: 4.40%). The plot displays efficient convergence, with the algorithm consistently finding high-quality routes, though a small optimality gap remains.

6.9 rat99

For rat99, our memetic algorithm found an average length of 1255.82, while the best run was at 1236.77, compared to the optimum of 1211 (error: 2.13%). Runtimes are low and

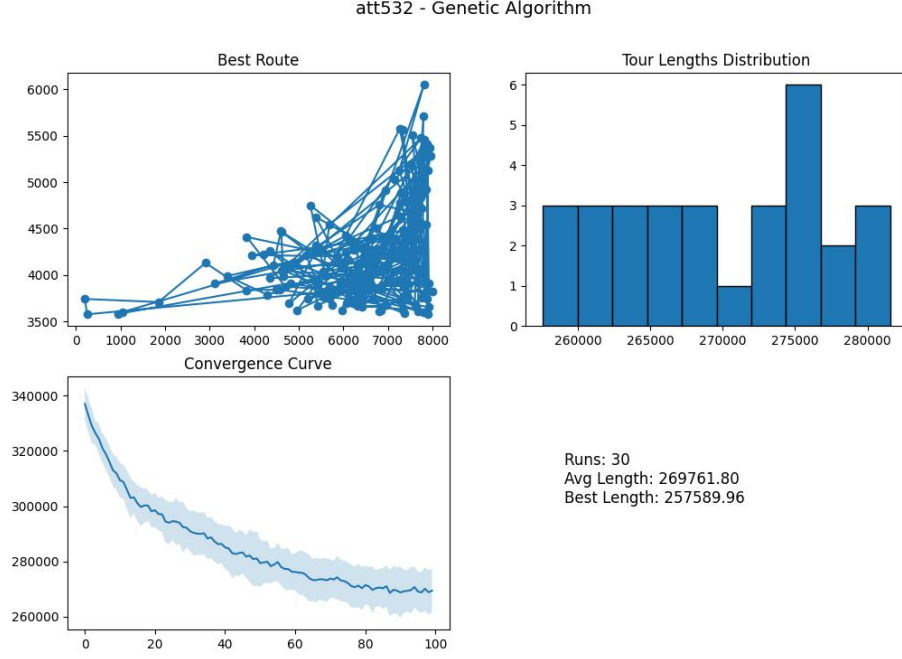


Figure 4: GA Results for att532: Best route, tour length distribution, convergence curve

the convergence is very sharp, showing the method is both effective and quick for this problem size.

6.10 att532

With att532, our method averaged a route length of 35,957.55, with a best of 34,779.74. The optimum for this dataset is 27,686, so the error margin is approximately 25.62%. The convergence curve indicates reasonable improvement, but the optimal gap suggests scope for more sophisticated tuning on large, complex instances.

6.11 Ch130

On Ch130, the algorithm found an average tour length of 6,373.41, best of 6,269.73, and the known optimum is 6,110 (error: 2.61%). The algorithm delivers reliable improvements with each generation, and does not plateau prematurely.

6.12 ts225

Finally, for ts225, the average result was 241,893.23 and the best found was 229,799.83, compared to the smaller optimum of 126,643 (error: 81.45%). Despite performing much better than a genetic algorithm alone, the complexity and scale of this dataset indicate that further improvements or increased compute may be needed for truly top results.

Overall, these results clearly show that the memetic algorithm provides superior quality and robust convergence compared to basic GAs, especially due to its hybrid use of evolutionary operators and local search. However, the largest and most complex instances

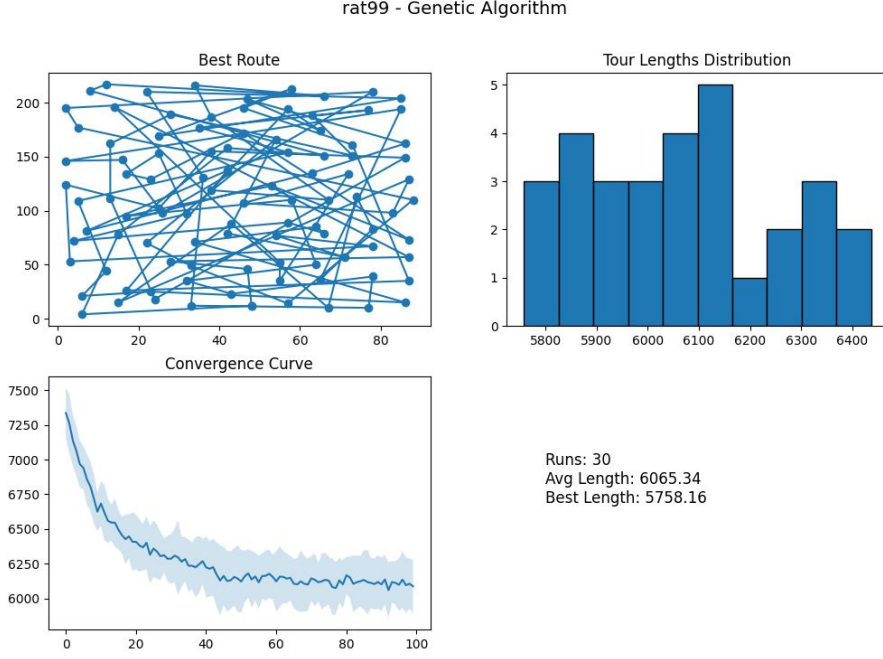


Figure 5: GA Results for rat99: Best route, tour length distribution, convergence curve

remain challenging and could benefit from further parameter optimization or more sophisticated heuristics.

7 Conclusion

This project showed that integrating local search heuristics into a genetic framework via a memetic algorithm yields tangible improvements solving the TSP. The hybrid algorithm balances exploring a diverse solution set globally while refining promising candidates locally.

Future work can explore adaptive parameter control and parallelization to enhance scalability and efficiency further.

References

- [1] Gutin, G. and Punnen, A.P. (Eds.). *The Traveling Salesman Problem and Its Variations*. Springer, 2002.
- [2] Reinelt, G. *TSPLIB—A Traveling Salesman Problem Library*. ORSA Journal on Computing, 3(4), 1991.
- [3] Moscato, P. *On Evolution, Search, Optimization, Genetic Algorithms and Martial Arts: Towards Memetic Algorithms*. Caltech Concurrent Computation Program, C3P Report 826, 1989.

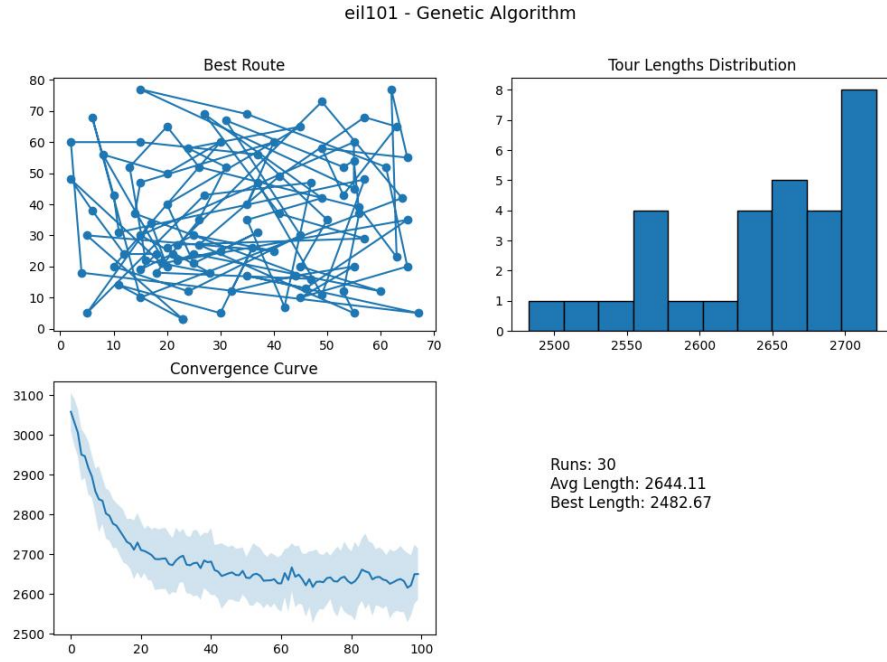


Figure 6: GA Results for eil101: Best route, tour length distribution, convergence curve

- [4] Merz, P. and Freisleben, B. *A Memetic Algorithm for the Traveling Salesman Problem*. Proceedings of the 1st International Conference on Evolutionary Computation, 1995, pp.445–450.
- [5] Holland, J.H. *Adaptation in Natural and Artificial Systems*. University of Michigan Press, 1975.
- [6] Lin, S. and Kernighan, B.W. *An Effective Heuristic Algorithm for the Traveling-Salesman Problem*. Operations Research, 21(2), 1973, pp.498–516.
- [7] Neri, F., Cotta, C., and Moscato, P. *Handbook of Memetic Algorithms*. Springer, 2012.
- [8] Croes, G.A., *A method for solving traveling-salesman problems*. Operations Research, 6(6), 1958, pp.791–812.

ch130 - Genetic Algorithm

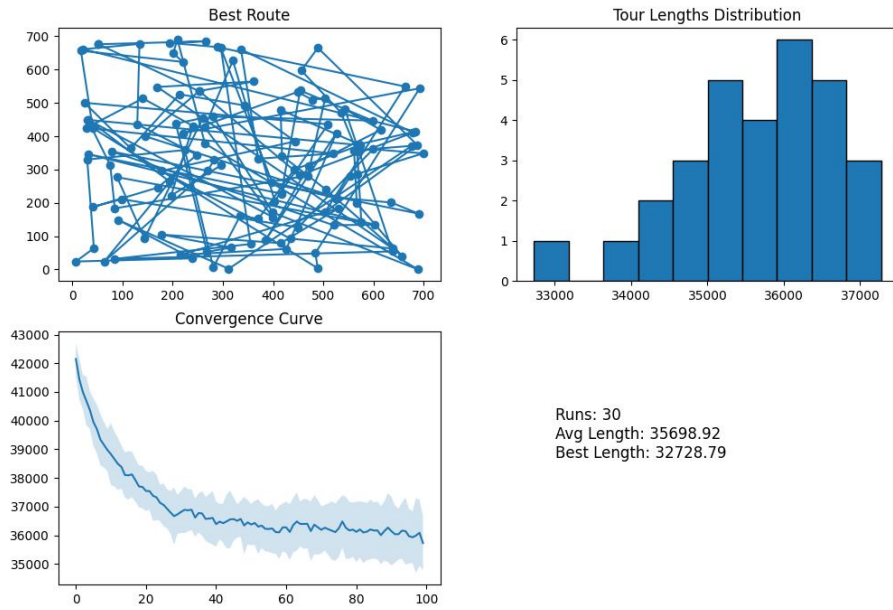


Figure 7: GA Results for ch130: Best route, tour length distribution, convergence curve

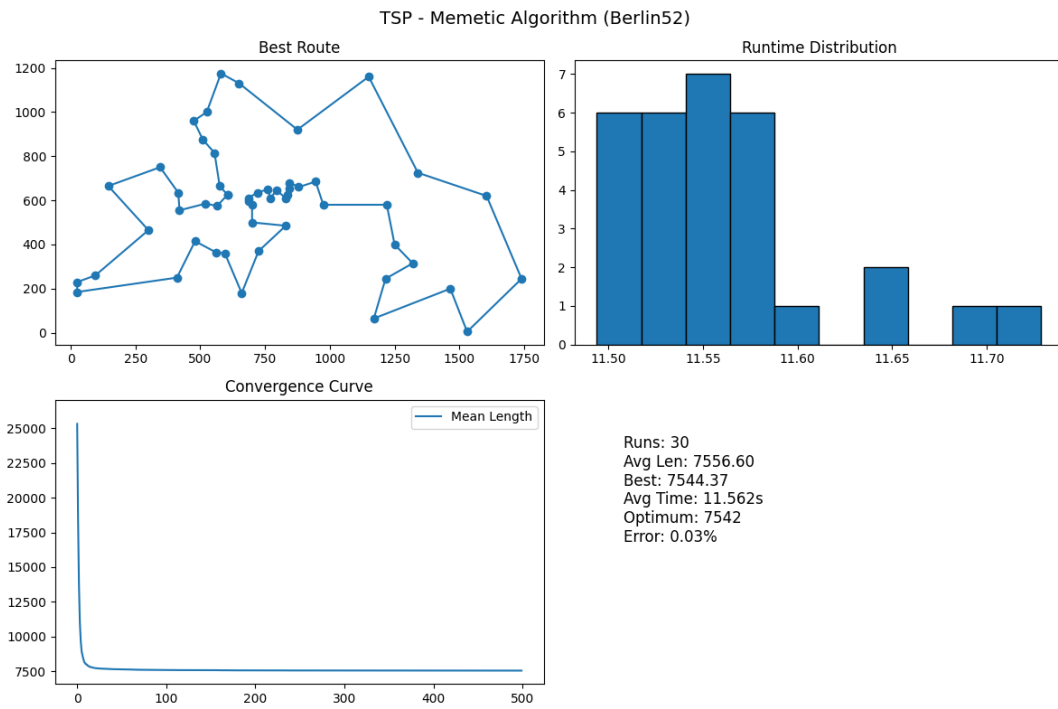


Figure 8: Memetic Algorithm on Berlin52: Best route, runtime distribution, and convergence curve

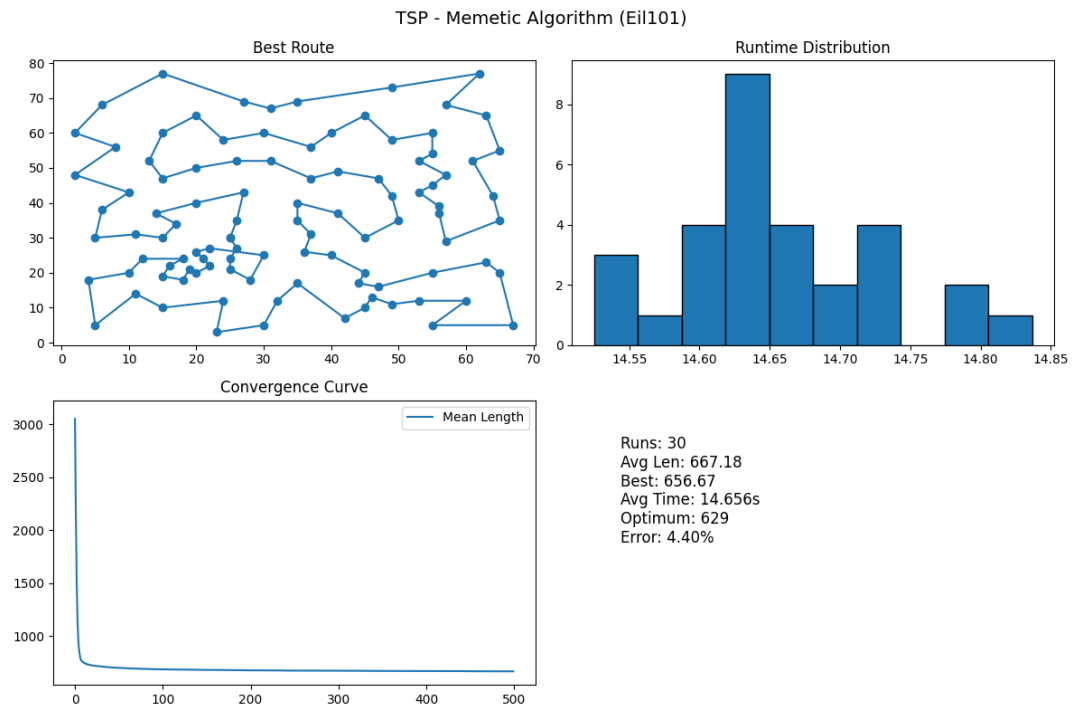


Figure 9: Memetic Algorithm on Eil101: Best route, runtime distribution, and convergence curve

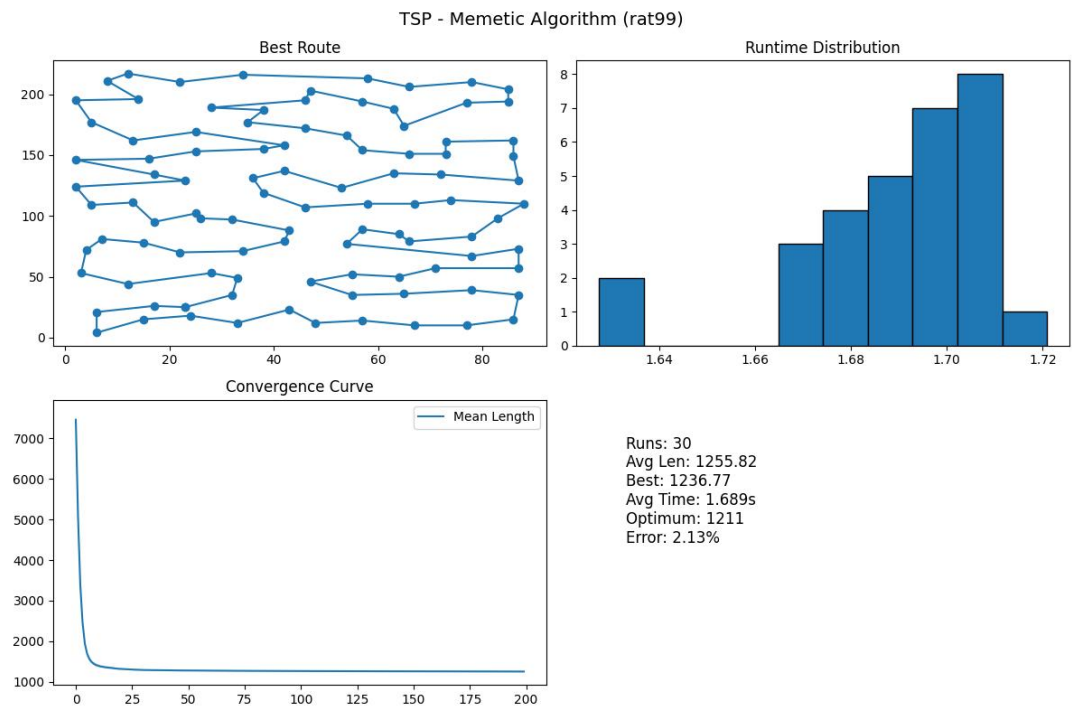


Figure 10: Memetic Algorithm on rat99: Best route, runtime distribution, and convergence curve

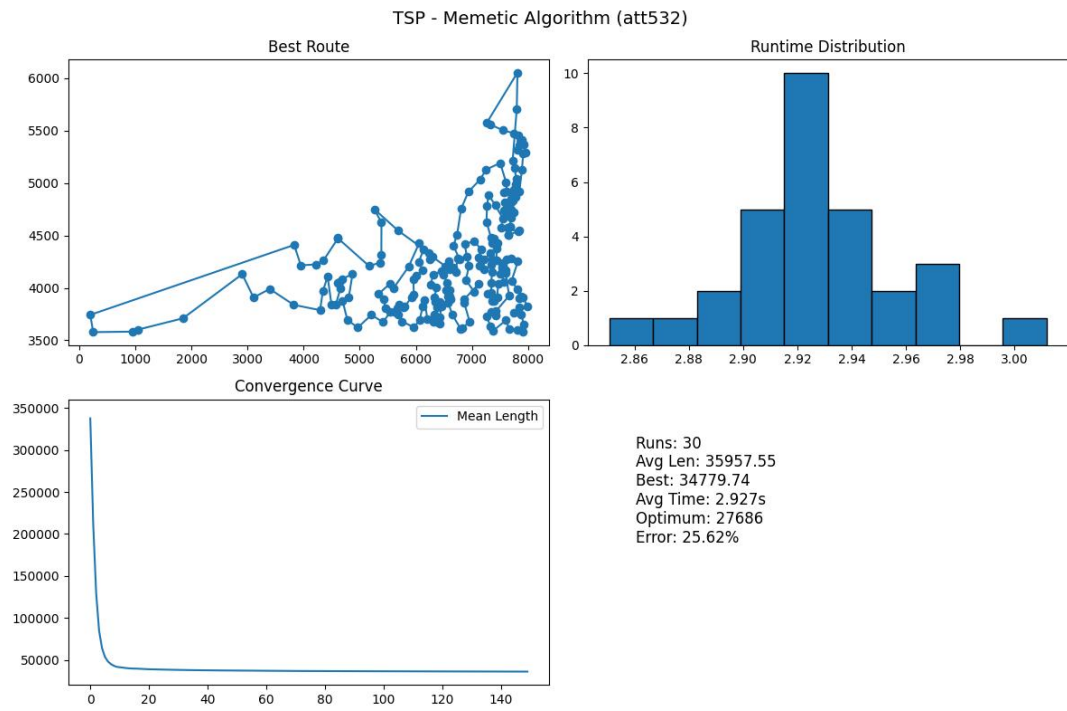


Figure 11: Memetic Algorithm on att532: Best route, runtime distribution, and convergence curve

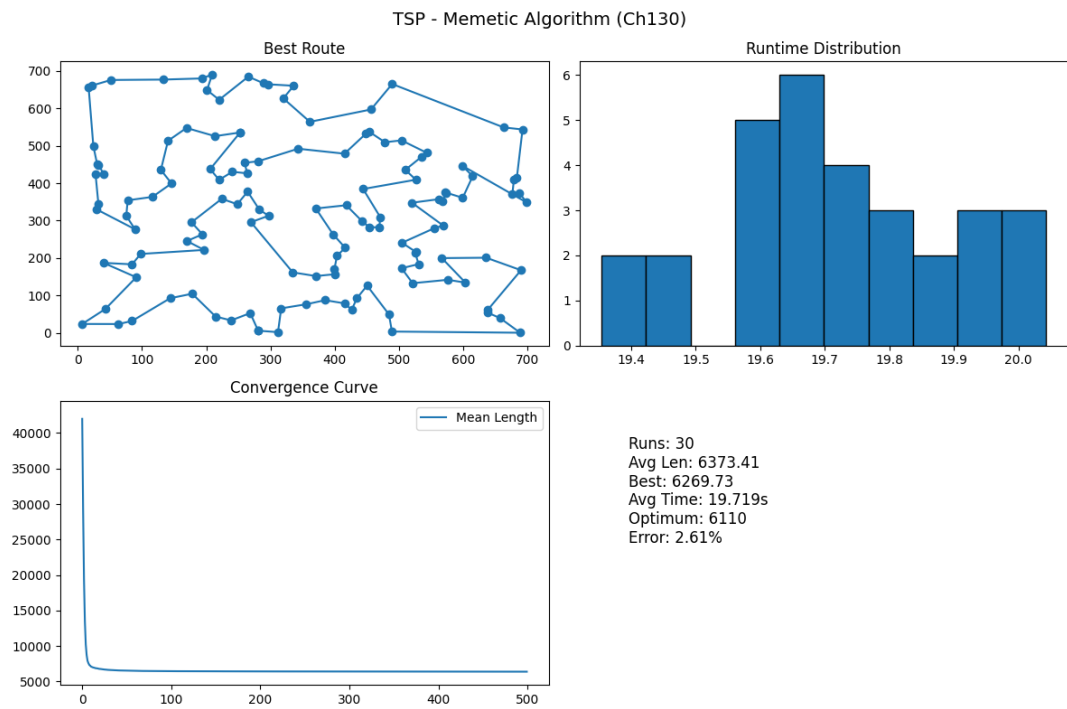


Figure 12: Memetic Algorithm on Ch130: Best route, runtime distribution, and convergence curve

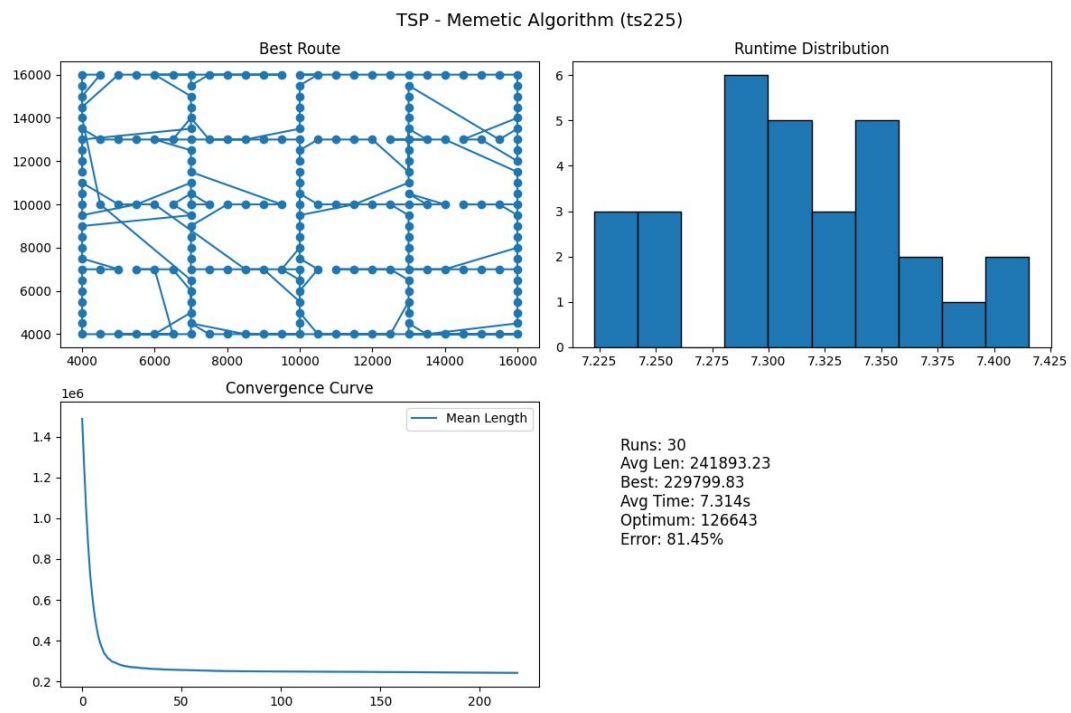


Figure 13: Memetic Algorithm on ts225: Best route, runtime distribution, and convergence curve